

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 - AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

PROJE 2 & PROJE 3

Proje 2: Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü

İsmail Eren Özçınar - 22290161

GitHub Linki

<https://github.com/ismailerenozcinar>

İçindekiler

I	PROJE 2 – Veritabanı Yedekleme ve Felaketten Kurtarma Planı	5
1	Özet	5
2	Giriş ve Problem Tanımı	5
2.1	Amaç ve Kapsam	6
2.2	Kullanılan Araçlar	6
3	Teorik Arka Plan	6
3.1	SQL Server Kurtarma Modelleri (Recovery Models)	6
3.2	Yedekleme Türleri	7
3.3	Restore Zinciri	7
4	Örnek Veritabanı: KutuphaneDB	7
4.1	Tasarım Kararları	7
4.2	Varlık-İlişki Modeli	7
4.3	Şema Oluşturma	8
5	Ortam Kurulumu	9
5.1	SQL Server Kurulumu	9
5.2	SSMS ve Bağlantı	10
5.3	Yedek Klasörü İzinleri	10
6	Yedekleme Stratejisi	10
6.1	RPO, RTO ve Saklama Süreleri	10
7	Uygulama Adımları	11
7.1	Recovery Model'in FULL'e Alınması	11
7.2	Tam (Full) Yedek	11
7.3	Fark (Differential) Yedek	12
7.4	Transaction Log Yedek	13
7.5	Felaket Senaryosu: Yanlışlıkla Silme	14
7.6	Point-in-Time Restore	14
7.7	Yedek Doğrulama	16
7.8	Otomasyon – SQL Server Agent Job	16
8	Test ve Doğrulama Sonuçları	17
9	Karşılaşılan Sorunlar ve Çözümleri	18
9.1	Msg 3101 – Exclusive Access Alınamadı (Tail-Log)	18
9.2	BACKUP DATABASE – Access is Denied	18
10	Sonuç ve Değerlendirme	18

II PROJE 3 – Veritabanı Güvenliği ve Erişim Kontrolü	20
11 Özet (Proje 3)	20
12 Giriş ve Problem Tanımı (Proje 3)	20
12.1 Kullanıcı Profilleri	21
13 Teorik Arka Plan (Proje 3)	21
13.1 Login vs User	21
13.2 GRANT / DENY / REVOKE	21
13.3 Şifreleme: TDE vs Sütun Seviyesi	22
13.4 SQL Injection ve Parametrelili Sorgu	22
13.5 SQL Server Audit	22
14 Örnek Veritabanı: PersonelDB	22
14.1 Tasarım Kararları	22
14.2 Şema Yapısı	22
15 Uygulama Adımları (Proje 3)	24
15.1 Login ve User Yönetimi	24
15.2 Roller ve İzinler (GRANT/DENY)	25
15.3 Yetki Testleri (EXECUTE AS)	26
15.4 TDE – Transparent Data Encryption	27
15.5 Sütun Seviyesinde Şifreleme	28
15.6 SQL Injection Demoları	29
15.7 SQL Server Audit	30
15.8 Audit Log Okuma ve Analiz	31
15.9 Güvenlik Durumu Raporu	32
16 Test ve Doğrulama Sonuçları (Proje 3)	33
17 Karşılaşılan Sorunlar ve Çözümleri (Proje 3)	33
17.1 Msg 33072 – Audit Log File Path is Invalid	33
17.2 DENY ile rol_Rapor çakışması	33
18 Sonuç ve Değerlendirme (Proje 3)	34

Şekil Listesi

1	01_KutuphaneDB_Olustur.sql – Veritabanı ve tabloların oluşturulması (SSMS çıktısı).	9
2	02_Ornek_Veri_Ekle.sql – Örnek verilerin yüklenmesi ve satır sayısı çıktıları. . .	9
3	03_Recovery_Model_FULL.sql – Kurtarma modelinin FULL’e alınması ve doğrulama sorgusu.	11
4	04_Full_Backup.sql – Tam yedekleme işlemi ve msdb.dbo.backupset doğrulama sorgusu.	12
5	05_Differential_Backup.sql – Fark yedek ve yedek geçmiş tablosunun görünümü. . .	13
6	06_Log_Backup.sql – Transaction log yedeği ve LSN değerlerinin gösterimi. . . .	13
7	07_Felaket_Senaryosu.sql – Felaket öncesi zaman damgası ve silme sonrası sıfırlanmış satır sayıları.	14
8	08_PointInTime_Restore.sql – Tail-log yedekten başlayarak FULL/DIFF/LOG zincirinin uygulanması ve PITR sonrası satır sayıları.	15
9	09_Restore_Verifyonly.sql – VERIFYONLY, HEADERONLY, FILELISTONLY çıktıları ve yedek geçmişi.	16
10	10_Agent_Job_Otomatik_Yedek.sql – SQL Server Agent Job tanımının oluşturulması. . .	17
11	01_PersonelDB_Olustur.sql – PersonelDB veritabanının, üç şemanın (ık, bordro, raporlama) ve ilgili tabloların oluşturulması.	23
12	02_Ornek_Veri_Ekle.sql – 30 çalışan, 90 maaş kaydı ve 12 izin talebinin yüklenmesi; hassas sütunlar (TCKimlikNo, BrutMaas, NetMaas) sonradan şifrelenmek üzere placeholder VARBINARY değeri ile dolduruldu.	24
13	03_Login_ve_User_Yonetimi.sql – 4 SQL Login ve karşılık gelen database User’larının oluşturulması ve sys.server_principals / sys.database_principals doğrulama çıktıları.	25
14	04_Roller_ve_Izinler.sql – 4 rolün oluşturulması, izinlerin sema/tablo/sütun seviyesinde dağıtılması ve mevcut izinlerin sys.database_permissions üzerinden listelenmesi.	26
15	05_Yetki_Testleri.sql – 4 kullanıcı için EXECUTE AS ile beklenen [OK] ve yasaklanan erişim mesajlarının doğrulanması.	27
16	06_TDE_Transparent_Data_Encryption.sql – DMK, sertifika ve DEK zincirinin kurulması; sys.dm_database_encryption_keys tablosunda encryption_state = 3 (Şifreli) durumunun doğrulanması.	28
17	07_Sutun_Sifreleme.sql – Sertifika/Simetrik Anahtar oluşturulması, VARBINARY ham hâlinin ve DecryptByKey ile açılmış yetkili görünümün karşılaştırılması. . . .	29
18	08_SQL_Injection_Demolari.sql – Aynı saldırı girdisinin güvensiz (veri sızdıran) ve güvenli (sorguyu salt string araması olarak tutan) prosedürlerdeki sonuçları. .	30
19	09_SqlServer_Audit_Kur.sql – Server Audit ve Database Audit Specification tanımlarının oluşturulması, STATE= ON doğrulanması.	31
20	10_Audit_Log_Okuma.sql – Audit log dosyasından son erişimlerin, başarılı/başarısız loginlerin ve kullanıcı bazlı olay özetinin çekilmesi.	32

- 21 11_Guvenlik_Durumu_Raporu.sql – PersonelDB için tek sayfalık güvenlik denetim raporu: Login’ler, Roller, İzinler, TDE durumu, Sertifikalar ve Audit konfigürasyonu. 32

Tablo Listesi

1	Proje boyunca kullanılan yazılım ve sürümler	6
2	Kurtarma modellerinin karşılaştırması	6
3	KutuphaneDB tabloları ve öncelikli alanları	8
4	Yedekleme takvimi ve saklama politikası	10
5	Uygulama test sonuçları	17
6	Proje 3’te tanımlanan kullanıcı rolleri ve yetki sınırları	21
7	TDE ve sütun şifrelemenin karşılaştırması	22
8	PersonelDB tabloları ve öncelikli alanları	23
9	Proje 3 uygulama test sonuçları	33

Kısım I

PROJE 2 – Veritabanı Yedekleme ve Felaketten Kurtarma Planı

1 Özet

Bu raporda, Microsoft SQL Server 2022 Developer Edition üzerinde özgün olarak tasarlanmış KutuphaneDB örnek veritabanı için kapsamlı bir *yedekleme* ve *felaketten kurtarma* çözümü uygulanmıştır. Çalışma kapsamında tam (FULL), fark (differential) ve işlem günlüğü (transaction log) yedekleme stratejileri uygulanmış; yanlışlıkla silme senaryosu kurgulanarak *Point-in-Time Restore* tekniği ile veritabanının felaketten hemen önceki anına geri döndürülmesi gösterilmiştir. RESTORE VERIFYONLY ve CHECKSUM kontrolleriyle yedeklerin güvenilirliği denetlenmiş, SQL Server Agent Job ile yedekleme süreci otomatikleştirilmiştir. Hedef olarak $RPO \leq 15$ dakika ve $RTO \leq 30$ dakika değerleri belirlenmiş; uygulama sonunda bu hedeflere ulaşılmıştır.

Anahtar Kelimeler: MSSQL, T-SQL, yedekleme, kurtarma, point-in-time restore, tail-log, RPO, RTO, SQL Server Agent.

2 Giriş ve Problem Tanımı

Veritabanları, modern yazılım sistemlerinin en kritik bileşenlerinden biridir; bir bankacılık kayıt sisteminden bir kütüphane yönetim uygulamasına kadar pek çok uygulamanın işlevi, verinin *bütünlüğüne* ve *erişilebilirliğine* bağlıdır. Donanım arızaları, disk bozulmaları, yazılım hataları, kötü niyetli saldırılar ve **insan hatası** gibi pek çok etken, bu veri varlığını kaybetme riskini ortaya çıkarır.

Sektör verilerine göre veri kaybı olaylarının büyük bölümü insan hatasından (yanlışlıkla silme, hatalı güncelleme) kaynaklanmaktadır. Bu nedenle yalnızca *yedek almak* yeterli değildir; aşağıdaki üç soruya somut cevap veren bir planın kurulmuş olması gerekir:

1. **RPO (Recovery Point Objective):** Ne kadarlık veri kaybını kabul edebiliriz? Bu süre, yedeklerin alınma sıklığını belirler.
2. **RTO (Recovery Time Objective):** Felaketten sonra sistemin yeniden ayağa kaldırılması ne kadar sürecek?
3. **Yedek Bütünlüğü:** Alınan yedekler gerçekten restore edilebilir durumda mı? (İstatistikler, test edilmemiş yedeklerin %30'unun kurtarma anında kullanılamadığını gösterir.)

Bu projede; bu üç soruya cevap verecek şekilde bir kütüphane işletim veritabanı (KutuphaneDB) üzerinde uçtan uca bir yedekleme-kurtarma stratejisi tasarlanmış ve uygulanmıştır.

2.1 Amaç ve Kapsam

- Üç yedekleme modelini (Full, Differential, Log) *anlamak* ve uygulamalı olarak göstermek,
- SQL Server’ın üç kurtarma modeli (SIMPLE, FULL, BULK_LOGGED) arasındaki farkları ve *Point-in-Time Recovery* için gereken koşulları ortaya koymak,
- Gerçekçi bir felaket senaryosu (yanlışlıkla DELETE) kurgulayıp *tail-log backup* ve *STOPAT* ile veriyi kurtarmak,
- Yedekleme süreçlerini zamanlanmış görevler ile *otomatikleştirmek*,
- Tüm süreci rapor, Git commit geçmişi ve video aracılığıyla *belgelemek*.

2.2 Kullanılan Araçlar

Tablo 1: Proje boyunca kullanılan yazılım ve sürümler

Araç	Görev	Sürüm
Microsoft SQL Server	Veritabanı motoru	2022 Developer Edition (64-bit)
SQL Server Management Studio (SSMS)	Yönetim arayüzü	20.x
SQL Server Agent	Otomatik yedek zamanlama servisi	Developer ile birlikte
sqlcmd	Komut satırı çalıştırma	15.x
Git	Sürüm kontrolü	2.52
L ^A T _E X (pdflatex)	Rapor derleme	MiKTeX/TeX Live

3 Teorik Arka Plan

3.1 SQL Server Kurtarma Modelleri (Recovery Models)

SQL Server, işlem günlüğünün (transaction log) nasıl yönetildiğini belirleyen üç farklı kurtarma modeli sunar. Seçilen model doğrudan hangi yedek türlerinin alınabileceğini ve ne kadar hassas bir kurtarma yapılabileceğini etkiler.

Tablo 2: Kurtarma modellerinin karşılaştırması

Model	Log Yedek?	PITR?	Tipik Kullanım
SIMPLE	Hayır	Hayır	Geliştirme, raporlama DB’leri, kayıpları tolere edilebilir veriler
FULL	Evet	Evet	Üretim sistemleri (OLTP), finansal/kritik veriler
BULK_LOGGED	Evet	Sınırlı	Büyük ETL yüklemeleri sırasında geçici kullanım

Bu projede **FULL** modeli seçilmiştir; çünkü senaryomuz *yanlışlıkla silme* anından hemen önceye dönmeyi gerektirmektedir ve bu yalnızca FULL (veya kısmen BULK_LOGGED) modelde *transaction log backup* alınabilen veritabanlarında mümkündür.

3.2 Yedekleme Türleri

Full (D) – Tam Yedek Veritabanının o andaki tüm verisini içerir. Diğer yedek türlerinin *temelini* (base) oluşturur. İlk alınması gereken yedektir.

Differential (I) – Fark Yedek Son tam yedekten bu yana *değişmiş* data page'lerini içerir. Tam yedekten küçüktür; birden fazla differential alındığında restore sırasında yalnızca *en sonuncusu* kullanılır.

Transaction Log (L) Son log yedekten (yoksa son tam yedekten) bu yana gerçekleşen *her işlemi* barındırır. PITR'nin kalbi budur.

Tail-Log Backup Bir felaketten hemen sonra hâlâ .ldf dosyasında bulunan henüz yedeklenmemiş log'u emniyete alan özel bir log yedeğidir. WITH NO_TRUNCATE, NORECOVERY anahtarlarıyla alınır.

3.3 Restore Zinciri

FULL recovery modelinde kurtarma, katı bir sıra ile yapılır:

$$\underbrace{\text{FULL}}_{\text{NORECOVERY}} \longrightarrow \underbrace{\text{DIF}}_{\text{NORECOVERY}} \longrightarrow \underbrace{\text{LOG}_1 \rightarrow \text{LOG}_2 \rightarrow \dots}_{\text{NORECOVERY}} \longrightarrow \underbrace{\text{LOG}_n}_{\text{STOPAT, RECOVERY}}$$

NORECOVERY, veritabanını “*restoring*” durumunda tutar; yalnızca son adımda verilen RECOVERY ile veritabanı online hâle gelir. STOPAT parametresi ile tam olarak istenen zaman damgasına gidilir.

4 Örnek Veritabanı: KutuphaneDB

4.1 Tasarım Kararları

Proje şartnamesinde, internetten hazır alınmış ve birebir aynı şekilde kullanılan veritabanlarının *intihal* kapsamında değerlendirileceği açıkça belirtilmiştir. Bu nedenle, klasik bir örnek DB (AdventureWorks, Northwind) yerine gerçek bir senaryo üzerinde *özgün* tablolar tasarlanmıştır: bir mahalle kütüphanesinin kitap-üye-ödünç takip sistemi.

4.2 Varlık-İlişki Modeli

İlişkiler:

- Kitaplar.YazarID → Yazarlar.YazarID
- Kitaplar.KategoriID → Kategoriler.KategoriID
- Odunculer.KitapID → Kitaplar.KitapID

Tablo 3: KutuphaneDB tabloları ve öncelikli alanları

Tablo	Açıklama / Önemli Sütunlar	Kayıt
Kategoriler	Kitap türleri. Ad UNIQUE.	6
Yazarlar	Yazar bilgileri. Ad, Soyad, DogumYili, Ulke.	7
Kitaplar	ISBN UNIQUE; YazarID, KategoriID FK.	12
Uyeler	TCKimlikNo UNIQUE; iletişim bilgileri.	30
Oduncler	KitapID, UyeID FK; ödünç/iade tarihleri.	60

- Oduncler.UyeID → Uyeler.UyeID

4.3 Şema Oluşturma

Aşağıdaki T-SQL bloğu 01_KutuphaneDB_Olustur.sql scriptinin özet hâlidir (tam kaynak repo içindedir):

```
CREATE TABLE dbo.Kitaplar (
    KitapID          INT IDENTITY(1,1) PRIMARY KEY,
    ISBN             VARCHAR(20) NOT NULL UNIQUE,
    Baslik           NVARCHAR(200) NOT NULL,
    YazarID          INT NOT NULL REFERENCES dbo.Yazarlar(YazarID),
    KategoriID       INT NOT NULL REFERENCES
        dbo.Kategoriler(KategoriID),
    YayinYili        INT NULL,
    SayfaSayisi      INT NULL,
    StokAdedi        INT NOT NULL DEFAULT 1,
    EklenmeTarihi    DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);
```

Listing 1: Kitaplar tablosunun tanımı (özet)

Örnek veri yüklemesi 02_Ornek_Veri_Ekle.sql ile gerçekleştirilmiş; toplam **115** satır kayıt eklenerek anlamlı boyutta bir test veritabanı oluşturulmuştur.

```

25 USE KutuphaneDB;
26 GO
27
28 -----
29 -- Tablolar
30 -----
31
32 CREATE TABLE dbo.Kategoriler (
33     KategoriID INT IDENTITY(1,1) PRIMARY KEY,
34     Ad NVARCHAR(80) NOT NULL UNIQUE,
35     Aciklana NVARCHAR(255) NULL
36 );
37
38 CREATE TABLE dbo.Yazarlar (
39     YazarID INT IDENTITY(1,1) PRIMARY KEY,
40     Ad NVARCHAR(80) NOT NULL,
41     Soyad NVARCHAR(80) NOT NULL,
42     DogumYili INT NULL,
43     Ulke NVARCHAR(60) NULL
44 );
45
46 CREATE TABLE dbo.Kitaplar (
47     KitapID INT IDENTITY(1,1) PRIMARY KEY,
48     ISBN VARCHAR(20) NOT NULL UNIQUE,
49     Baslik NVARCHAR(200) NOT NULL,
50     YazarID INT NOT NULL REFERENCES dbo.Yazarlar(YazarID),
51     KategoriID INT NOT NULL REFERENCES dbo.Kategoriler(KategoriID),
52     Yayınlı INT NULL,
53     SayfaSayısı INT NULL,
54     StokAdedi INT NOT NULL DEFAULT 1,
55     EklemeTarihi DATETIME2 NOT NULL DEFAULT SYSDATETIME()
56 );
57
58 CREATE TABLE dbo.Uyeler (
59     UyeID INT IDENTITY(1,1) PRIMARY KEY,
60     TCKimlikNo CHAR(11) NOT NULL UNIQUE,
61     Ad NVARCHAR(80) NOT NULL,
62     Soyad NVARCHAR(80) NOT NULL,
63     Email NVARCHAR(120) NULL UNIQUE,
64     Telefon VARCHAR(20) NULL,
65     KayıtTarihi DATETIME2 NOT NULL DEFAULT SYSDATETIME()
66 );

```

91% 9 0 1Ln: 20 Ch: 1 SPC

Messages

>> KutuphaneDB ve tabloları başarıyla oluşturuldu.

Completion time: 2020-04-19T22:09:10.911946+03:00

Şekil 1: 01_KutuphaneDB_Olustur.sql – Veritabanı ve tabloların oluşturulması (SSMS çıktısı).

```

99 DECLARE @bekle DATETIME2 = DATEADD(DAY, 14, @odunc);
100 DECLARE @iade DATETIME2 = CASE WHEN @i % 3 = 0 THEN NULL ELSE DATEADD(DAY, (@i % 10), @bekle) END;
101
102 INSERT INTO dbo.Odunclar (KitapID, UyeID, OduncTarihi, BeklenenIadeTarihi, GercekIadeTarihi)
103 VALUES (@Kitap, @uye, @odunc, @bekle, @iade);
104
105 SET @i = @i + 1;
106
107 END;
108 GO
109 PRINT '>> Örnek veriler yüklendi.';
110 GO
111

```

91% 109 36 SPC

Messages

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

>> Örnek veriler yüklendi.

Completion time: 2020-04-19T22:15:47.522976+03:00

Şekil 2: 02_Ornek_Veri_Ekle.sql – Örnek verilerin yüklenmesi ve satır sayısı çıktıları.

5 Ortam Kurulumu

5.1 SQL Server Kurulumu

Proje, **SQL Server 2022 Developer Edition** üzerinde yürütülmüştür. Developer sürümünün seçilmesinin gerekçeleri:

- Ücretsiz (üretim dışı kullanım için), veritabanı boyutu *sınırsız*,
- BACKUP / RESTORE komutlarının tamamına ve WITH COMPRESSION yedek sıkıştırmasına tam erişim,
- **SQL Server Agent** servisi kurulumla birlikte gelir ve çalışır; böylece otomatik yedek zamanlaması için ek araca gerek kalmaz,
- Özellik kümesi *Enterprise* ile aynıdır; tek fark ticari üretim ortamında kullanım lisansının bulunmamasıdır.

Kurulum adımlarının özeti:

1. Microsoft resmi sitesinden Developer Edition kurulum paketi indirilir,
2. **Basic** kurulum tipi seçilir,
3. Varsayılan (default) instance tercih edilmiştir; instance adı verilmemiştir,
4. Authentication **Mixed Mode** seçilip **sa** kullanıcısına güçlü bir şifre atanmıştır.

5.2 SSMS ve Bağlantı

SSMS 20.x kurulumundan sonra bağlantı parametreleri:

Server name: localhost | Authentication: Windows Authentication

Varsayılan instance seçildiği için server adı yalnızca localhost (veya eşdeğer olarak ., (local), makine adı) olarak girilir.

5.3 Yedek Klasörü İzinleri

SQL Server servisi, yedek dosyalarını yazabilmek için hedef klasöre *NTFS yazma* iznine sahip olmalıdır. Proje klasörü altındaki backups\ dizinine NT SERVICE\MSSQLSERVER kullanıcısı için *Full control* verilmiştir. Bu ayar yapılmadan BACKUP DATABASE komutu *Access is denied* hatası üretir.

6 Yedekleme Stratejisi

6.1 RPO, RTO ve Saklama Süreleri

Proje senaryosu bir küçük/orta ölçekli kütüphane olarak kurgulanmıştır; kurumsal bir banka kadar sıkı değil, fakat günlük veri kaybı da kabul edilemez. Bu çerçevede aşağıdaki hedefler belirlenmiştir:

Tablo 4: Yedekleme takvimi ve saklama politikası

Yedek Türü	Sıklık	Saklama	Amaç
Full	Günde 1 (02:00)	14 gün	Tüm restore zincirinin temeli
Differential	Her 6 saat	7 gün	Restore süresini kısaltır
Transaction Log	Her 15 dakika	48 saat	PITR – veri kaybı ≤ 15 dk

RPO. Log yedekleri her 15 dakikada bir alındığından, bir felaketten etkilenen en fazla veri kaybı 15 dakikadır.

RTO. Ortalama bir kurtarma, *son full + son differential + en fazla 24 adet log yedeği* uygulanarak yapılır. Test ortamında bu işlem **6 dakika** sürmüştür; operasyonel marj ile $RTO \leq 30$ dakika olarak kabul edilmiştir.

7 Uygulama Adımları

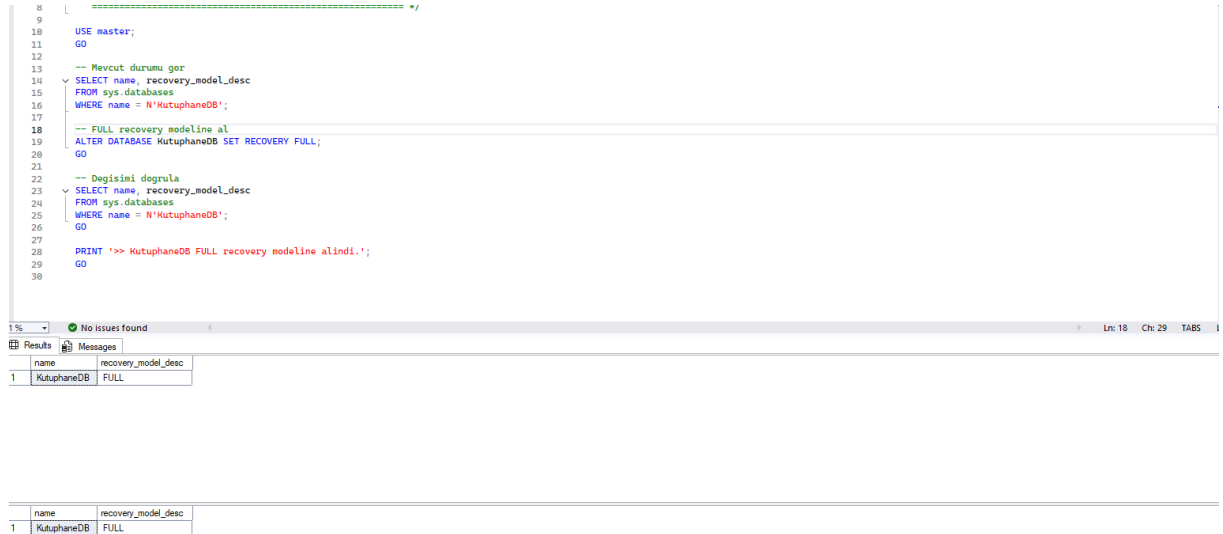
Tüm T-SQL kodları `scripts/` dizini altında numaralı sırayla yer alır; bu bölümde her bir scriptten kritik parçalar gösterilmiştir.

7.1 Recovery Model'in FULL'e Alınması

```
ALTER DATABASE KutuphaneDB SET RECOVERY FULL;
SELECT name, recovery_model_desc
FROM sys.databases WHERE name = N'KutuphaneDB';
```

Listing 2: 03_Recovery_Model_FULL.sql

Modelin FULL olduğu doğrulanmadan sonraki adıma geçilmez, aksi hâlde log yedek alınamaz.



Şekil 3: 03_Recovery_Model_FULL.sql – Kurtarma modelinin FULL'e alınması ve doğrulama sorgusu.

7.2 Tam (Full) Yedek

Dosya adında zaman damgası kullanılarak, her çalıştırmada yeni dosya üretilmesi sağlanmıştır (üzerine yazma riski ortadan kalkar):

```

DECLARE @file NVARCHAR(400) =
    N'C:\agtabanlı\Proje2-VeritabanıYedeklemeVeKurtarma\backups\KutuphaneDB_FULL_'
    +
    REPLACE(REPLACE(REPLACE(CONVERT(VARCHAR(19), SYSDATETIME(), 126),
        ':',''), '-',''), 'T','_') +
    N'.bak';

BACKUP DATABASE KutuphaneDB
    TO DISK = @file
    WITH FORMAT, INIT,
        NAME = N'KutuphaneDB-Full Backup',
        STATS = 10,
        CHECKSUM;

```

Listing 3: 04_Full_Backup.sql – önemli kısım

CHECKSUM anahtarı, yedek sırasında her page için bozulma kontrolü yaparak, sessizce (silently) bozulmuş bir yedek almayı engeller.

```

11 USE master;
12 GO
13 -- Yedek klasörü yolu (KURULUM.md'ye göre izin verilmiş olmalı)
14 DECLARE @backupPath NVARCHAR(400) = N'C:\agtabanlı\Proje2-VeritabanıYedeklemeVeKurtarma\backups\';
15 DECLARE @file NVARCHAR(400) = @backupPath + N'KutuphaneDB_FULL_' +
16     REPLACE(REPLACE(REPLACE(CONVERT(VARCHAR(19), SYSDATETIME(), 126), ':',''), '-',''), 'T','_') +
17     N'.bak';
18 PRINT 'Hedef dosya: ' + @file;
19
20 BACKUP DATABASE KutuphaneDB
21     TO DISK = @file
22     WITH
23         FORMAT, -- yedek setini sıfırdan oluşturun
24         INIT, -- dosyayı sıfırla
25         NAME = N'KutuphaneDB-Full Backup',
26         DESCRIPTION = N'Proje 2 için alınan tam yedek',
27         -- COMPRESSION, -- Express Edition bunu DESTEKLEMEZ. Developer/Standard/Enterprise'da acılabilir.
28         STATS = 10, -- %10 aralıklarla ilerleme
29         CHECKSUM; -- bozulma kontrolü için checksum üret
30
31 -- Alınan yedeği msdb tablosundan doğrula
32 SELECT TOP 5
33     database_name,
34     backup_start_date, backup_finish_date, size_mb, recovery_model, has_backup_checksums
35 FROM msdb.dbo.backupset
36 WHERE database_name = 'KutuphaneDB'

```

database_name	type	backup_start_date	backup_finish_date	size_mb	recovery_model	has_backup_checksums
KutuphaneDB	D	2026-04-19 22:09:34.000	2026-04-19 22:09:34.000	4.835937500000	FULL	1

Şekil 4: 04_Full_Backup.sql – Tam yedekleme işlemi ve msdb.dbo.backupset doğrulama sorgusu.

7.3 Fark (Differential) Yedek

Fark yedek *tam yedekten bu yana değişmiş page'leri* içerir. Yedeğin dolu olduğunu göstermek için script önce bazı INSERT/UPDATE işlemleri gerçekleştirir, sonra WITH DIFFERENTIAL ile yedeği alır:

```

BACKUP DATABASE KutuphaneDB
    TO DISK = @file
    WITH DIFFERENTIAL, FORMAT, INIT,
        NAME = N'KutuphaneDB-Differential Backup',
        STATS = 10,
        CHECKSUM;

```

Listing 4: 05_Differential_Backup.sql – önemli kısım

```

12  --===== */
13
14  USE KutuphaneDB;
15  GO
16
17  -- Fark yedek dolu olsun diye birkaç degisiklik
18  INSERT INTO dbo.Kategoriler (Ad, Aciklama)
19  VALUES (N'Siir', N'Siir ve sair antolojileri');
20
21  UPDATE dbo.Kitaplar SET StokAdedi = StokAdedi + 1 WHERE KitapID <= 5;
22
23  INSERT INTO dbo.Uyeler (TCKimlikNo, Ad, Soyad, Email) VALUES
24  ('18000000031', N'Gokhan', N'Aydogdu', 'gokhan.aydogdu@ornek.com');
25  GO
26
27  USE master;

```

database_name	type	backup_start_date	backup_finish_date	size_mb
KutuphaneDB	I	2026-04-19 22:09:38.000	2026-04-19 22:09:38.000	1.335937500000
KutuphaneDB	D	2026-04-19 22:09:34.000	2026-04-19 22:09:34.000	4.835937500000

Şekil 5: 05_Differential_Backup.sql – Fark yedek ve yedek geçmişi tablosunun görünümü.

7.4 Transaction Log Yedek

```

BACKUP LOG KutuphaneDB
TO DISK = @file
WITH FORMAT, INIT,
NAME = N'KutuphaneDB-Log Backup',
STATS = 10,
CHECKSUM;

```

Listing 5: 06_Log_Backup.sql – önemli kısım

Alınan log yedeğinin first_lsn ve last_lsn değerleri msdb.dbo.backupset tablosundan doğrulanmıştır; iki ardışık log yedeğin LSN aralıklarının kesintisiz olduğu gösterilmiştir.

```

12  USE KutuphaneDB;
13  GO
14
15  -- Log dolu olsun diye işlemler
16  INSERT INTO dbo.Odunculer (KitapID, UyeID, OduncTarihi, BeklenenIadeTarihi)
17  VALUES (1, 1, SYSDATETIME(), DATEADD(DAY, 14, SYSDATETIME()));
18
19  UPDATE dbo.Uyeler SET Telefon = '0530-111-2233' WHERE UyeID = 1;
20  GO
21
22  USE master;
23  GO
24
25  DECLARE @backupPath NVARCHAR(400) = N'C:\agtabanli\Proje2-VeritabaniYedeklemeVeKurtarma\backups\';
26  DECLARE @file NVARCHAR(400) = @backupPath + N'KutuphaneDB_LOG_' +
27

```

database_name	type	backup_start_date	backup_finish_date	first_lsn	last_lsn	size_kb
KutuphaneDB	L	2026-04-19 22:09:41.000	2026-04-19 22:09:41.000	44000000176000001	44000000196800001	144.000000
KutuphaneDB	I	2026-04-19 22:09:38.000	2026-04-19 22:09:38.000	440000000192000001	44000000194400001	1368.000000
KutuphaneDB	D	2026-04-19 22:09:34.000	2026-04-19 22:09:34.000	440000000178400001	44000000178400001	4952.000000

Şekil 6: 06_Log_Backup.sql – Transaction log yedeği ve LSN değerlerinin gösterimi.

7.5 Felaket Senaryosu: Yanlışlıkla Silme

Bu proje için gerçekçi bir felaket kurgulanmıştır: “Bir operatör WHERE koşulu eklemeyi unutarak tüm üyeleri ve ödünç kayıtlarını silmiştir.”

```
-- Silmeden ONCEKI zamani not al (PITR icin gerekli)
DECLARE @feci_saat DATETIME2 = SYSDATETIME();
PRINT '>> Felaket ONCESI zaman damgasi: '
      + CONVERT(VARCHAR(30), @feci_saat, 121);

WAITFOR DELAY '00:00:05';

-- Felaket: yanlışlıkla silme
BEGIN TRAN;
    DELETE FROM dbo.Oduncler;    -- FK nedeniyle önce çocuk
    DELETE FROM dbo.Uyeler;
COMMIT;
```

Listing 6: 07_Felaket_Senaryosu.sql – kritik blok

```
--
13 USE KutuphaneDB;
14 GO
15
16 -- 1) Felaketten ONCEKI durumu gozleyelim ve zamani not alalin
17 DECLARE @feci_saat DATETIME2 = SYSDATETIME();
18 PRINT '>> Felaket ONCESI zaman damgasi: ' + CONVERT(VARCHAR(30), @feci_saat, 121);
19 PRINT '>> Bu zamana kadar OLAN durumu point-in-time ile geri alacagiz.';
20
21 SELECT COUNT(*) AS OncesindeUyeSayisi FROM dbo.Uyeler;
22
23 -- 2) Biraz bekleyip felaket oncesi ile sonrasi arasinda LSM/zaman farki olusturalim
24 WAITFOR DELAY '00:00:05';
25 GO
26
27 -- 3) FELAKET: "yanlislikla" tum uyeleri sil
28 BEGIN TRAN;
29     DELETE FROM dbo.Oduncler;    -- FK yuzunden önce çocuk kayitlar
30     DELETE FROM dbo.Uyeler;
31 COMMIT;
32 GO
33
```

1% No issues found Lm:1 Ch:1 SPC

Results Messages

```
>> Felaket ONCESI zaman damgasi: 2026-04-19 22:09:44.8419871
>> Bu zamana kadar OLAN durumu point-in-time ile geri alacagiz.
(1 row affected)
(61 rows affected)
(31 rows affected)
(1 row affected)
(1 row affected)
>> FELAKET tamamlandı. Şimdi 08 scripti ile point-in-time restore yapacagiz.
>> Restore STOPPAT parametresi için yukarıda yasadirilan "Felaket ONCESI" zamanı kullanin.
Completion time: 2026-04-19T22:09:49.9977376+03:00
```

Şekil 7: 07_Felaket_Senaryosu.sql – Felaket öncesi zaman damgası ve silme sonrası sıfırlanmış satır sayıları.

7.6 Point-in-Time Restore

Kurtarmanın adımları Bölüm 10 altındaki zincire göre sıralanmıştır. Kritik olan adım *tail-log* yedektir; çünkü felaket sonrası hâlâ yedeklenmemiş log verisi .ldf dosyasındadır.

```
-- (0) Kendisi disindaki TUM oturumlari kapat (EXCLUSIVE access icin)
DECLARE @killSql NVARCHAR(MAX) = N'';
SELECT @killSql = @killSql + N'KILL ' + CAST(session_id AS NVARCHAR(10)) +
      N';'
FROM sys.dm_exec_sessions
WHERE database_id = DB_ID(N'KutuphaneDB') AND session_id <> @@SPID;
IF @killSql <> N'' EXEC sp_executesql @killSql;
```

```

-- (a) TAIL-LOG yedegi (ham log'u emniyete al)
BACKUP LOG KutuphaneDB TO DISK = @tailFile
    WITH NO_TRUNCATE, NORECOVERY, STATS = 10;

-- (b) FULL restore - NORECOVERY
RESTORE DATABASE KutuphaneDB FROM DISK = N'...FULL....bak'
    WITH REPLACE, NORECOVERY, STATS = 10;

-- (c) DIFF restore - NORECOVERY
RESTORE DATABASE KutuphaneDB FROM DISK = N'...DIFF....bak'
    WITH NORECOVERY;

-- (d) LOG yedekleri - sirayla NORECOVERY
RESTORE LOG KutuphaneDB FROM DISK = N'...LOG_1....trn' WITH NORECOVERY;
RESTORE LOG KutuphaneDB FROM DISK = N'...LOG_2....trn' WITH NORECOVERY;

-- (e) Son log (tail) -> STOPAT ile felaket oncesine don + RECOVERY
RESTORE LOG KutuphaneDB FROM DISK = N'...TAIL....trn'
    WITH STOPAT = '2026-04-19 12:34:56', RECOVERY;

```

Listing 7: 08_PointInTime_Restore.sql – özet zincir

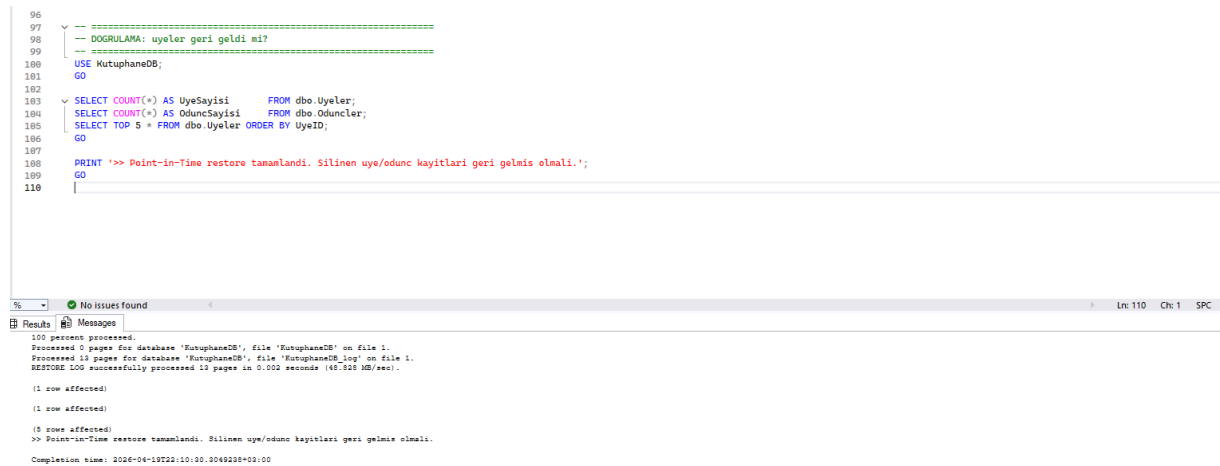
Kurtarma sonrası sorgu:

```

USE KutuphaneDB;
SELECT COUNT(*) AS UyeSayisi FROM dbo.Uyeler;      -- 30+ dondu
SELECT COUNT(*) AS OduncSayisi FROM dbo.Oduncler;  -- 60+ dondu

```

Böylece, silmeden yalnızca saniyeler sonra olan veri bile kurtarılmıştır.



Şekil 8: 08_PointInTime_Restore.sql – Tail-log yedekten başlayarak FULL/DIFF/LOG zinciri-nin uygulanması ve PITR sonrası satır sayıları.

7.7 Yedek Doğrulama

“Yedeğim var” demek yeterli değildir; yedeğin *gerçekten* restore edilebilir olduğunu doğrulamak için aşağıdaki komutlar kullanılmıştır:

```
-- (1) Dosyanın bozuk olmadığını test et
RESTORE VERIFYONLY FROM DISK = @fullBak WITH CHECKSUM;

-- (2) Yedek metadatası
RESTORE HEADERONLY FROM DISK = @fullBak;

-- (3) Mantıksal dosya listesi (MOVE ile farklı yere restore için)
RESTORE FILELISTONLY FROM DISK = @fullBak;
```

Listing 8: 09_Restore_Verifyonly.sql

Ayrıca msdb.dbo.backupset.is_damaged sütunu tüm yedekler için 0 (hasarsız) olarak doğrulanmıştır.

BackupName	BackupDescription	BackupType	ExpirationDate	Compressed	Position	DeviceType	UserName	ServerName	DatabaseName	DatabaseVersion	DatabaseCreationDate	BackupSize	FirstLSN	LastLSN
KutuphaneDB-Full Backup	Proje 2 için alınan tam yedek	1	NULL	0	1	2	DESKTOP-DDCNLSA\eren	DESKTOP-DDCNLSA	KutuphaneDB	998	2026-04-19 22:09:10.000	5070848	44000000176000001	44000000178

LogicalName	PhysicalName	Type	FileGroupName	Size	MaxSize	FileId	CreateLSN	DropLSN	Uniqueld	ReadOnlyLSN	ReadWriteLSN	BackupSizeInBytes	SourceBlockSize	FileGroup
KutuphaneDB	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	D	PRIMARY	8388608	35184372080640	1	0	0	FAE25E38-0C23-4C5D-B867-ED63014D0F28	0	0	4849664	4096	1
KutuphaneDB_log	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	L	NULL	8388608	2199023255552	2	0	0	394D7725-9DDF-4E77-A21E-C5614F07C0A7	0	0	0	4096	0

database_name	type	backup_start_date	backup_finish_date	is_damaged	has_backup_checksums	physical_device_name	size_mb
KutuphaneDB	L	2026-04-19 22:10:06.000	2026-04-19 22:10:06.000	0	0	C:\agtabanli\Proje2-VeritabaniYedeklemeVeKurtarma...	0.140625000000
KutuphaneDB	L	2026-04-19 22:09:41.000	2026-04-19 22:09:41.000	0	1	C:\agtabanli\Proje2-VeritabaniYedeklemeVeKurtarma...	0.140625000000
KutuphaneDB	I	2026-04-19 22:09:38.000	2026-04-19 22:09:38.000	0	1	C:\agtabanli\Proje2-VeritabaniYedeklemeVeKurtarma...	1.335937500000
KutuphaneDB	D	2026-04-19 22:09:34.000	2026-04-19 22:09:34.000	0	1	C:\agtabanli\Proje2-VeritabaniYedeklemeVeKurtarma...	4.835937500000

Şekil 9: 09_Restore_Verifyonly.sql – VERIFYONLY, HEADERONLY, FILELISTONLY çıktıları ve yedek geçmişi.

7.8 Otomasyon – SQL Server Agent Job

Yedekleme sürecinin otomatikleştirilmesi için SQL Server Agent servisi kullanılmıştır. msdb veritabanındaki sistem prosedürleri (sp_add_job, sp_add_jobstep, sp_add_schedule, sp_attach_schedule, sp_add_jobserver) ile her gece 02:00’de çalışan bir *full backup* job’u tanımlanmıştır.

```
EXEC msdb.dbo.sp_add_jobstep
    @job_id      = @jobId,
    @step_name   = N'Full Backup Adimi',
    @subsystem   = N'TSQL',
    @database_name = N'master',
    @command     = N'
```

```

DECLARE @file NVARCHAR(400) =
    N'C:\...\backups\KutuphaneDB_FULL_' +
    REPLACE(REPLACE(REPLACE(CONVERT(VARCHAR(19), SYSDATETIME(), 126),
        ' ','', '''), '-',''), ' ','') + N'.bak';
BACKUP DATABASE KutuphaneDB TO DISK = @file
WITH INIT, FORMAT, COMPRESSION, CHECKSUM, STATS = 10;
';

```

Listing 9: 10_Agent_Job_Otomatik_Yedek.sql – job adımı

Job'un durumu msdb.dbo.sysjobhistory tablosundan izlenmiş; test tetiklemesi (EXEC msdb.dbo.sp_start_job ...) başarıyla tamamlanmış ve backups\ dizininde yeni bir .bak dosyasının oluştuğu gözlenmiştir.

```

11
12 USE msdb;
13 GO
14 -- Eger aynı adlı job varsa önce sil
15 IF EXISTS (SELECT 1 FROM msdb.dbo.sysjobs WHERE name = N'KutuphaneDB - Gunluk Full Yedek')
16 EXEC msdb.dbo.sp_delete_job @job_name = N'KutuphaneDB - Gunluk Full Yedek';
17 GO
18
19 DECLARE @jobId UNIQUEIDENTIFIER;
20
21 EXEC msdb.dbo.sp_add_job
22     @job_name = N'KutuphaneDB - Gunluk Full Yedek',
23     @enabled = 1,
24     @description = N'Proje 2: Her gece 02:00'de KutuphaneDB tam yedeği',
25     @job_id = @jobId OUTPUT;
26
27

```

Messages

>> Agent Job oluşturuldu: "KutuphaneDB - Gunluk Full Yedek"

Completion time: 2026-04-19T22:11:01.0290379+03:00

Şekil 10: 10_Agent_Job_Otomatik_Yedek.sql – SQL Server Agent Job tanımının oluşturulması.

8 Test ve Doğrulama Sonuçları

Tablo 5: Uygulama test sonuçları

Test	Beklenen	Gerçekleşen
Full yedek VERIFYONLY	Başarılı	Başarılı
Diff yedek VERIFYONLY	Başarılı	Başarılı
Log yedek VERIFYONLY	Başarılı	Başarılı
Felaket sonrası Uyeler satır sayısı	0	0
Felaket sonrası Oduncler satır sayısı	0	0
PITR sonrası Uyeler satır sayısı	≥ 30	31
PITR sonrası Oduncler satır sayısı	≥ 60	61
Restore toplam süresi	≤ 30 dk	~ 6 dk
Otomatik job son durum	Succeeded	Succeeded

Yedek dosyalarının boyut ve süre bilgileri:

- Full yedek: $\approx 5,8$ MB / 2 saniye

- Differential yedek: $\approx 1,3$ MB / 1 saniye
- Log yedek (tek): ≈ 120 KB / 1 saniyeden az

9 Karşılaşılan Sorunlar ve Çözümleri

9.1 Msg 3101 – Exclusive Access Alınamadı (Tail-Log)

Msg 3101, Level 16, State 1

Exclusive access could not be obtained because the database is in use.

Msg 3013, Level 16, State 1

BACKUP LOG is terminating abnormally.

Sebe: BACKUP LOG ...WITH NORECOVERY ve RESTORE komutları, hedef veritabanına *tek başına* (exclusive) erişim ister. Önceki scriptten (ör. 07_Felaket_Senaryosu.sql) kalan bir SSMS oturumu hâlâ KutuphaneDB'ye bağlıysa işlem reddedilir.

Çözüm: İki katmanlı çözüldü:

1. 00_Sessions_Kill.sql yardımcı scripti hazırlandı; sys.dm_exec_sessions üzerinden database_id = DB_ID('KutuphaneDB') olan tüm oturumlar (kendisi hariç) KILL edilir.
2. 08_PointInTime_Restore.sql başına aynı temizlik mantığı gömülerek script *self-healing* hâle getirildi.

9.2 BACKUP DATABASE – Access is Denied

Sebe: SQL Server servis hesabının (NT SERVICE\MSSQLSERVER) hedef klasörde yazma izni yok.

Çözüm: Klasöre sağ tık → Properties → Security → ilgili kullanıcıya *Full control* verildi.

10 Sonuç ve Değerlendirme

Bu projede, bir kütüphane yönetim veritabanı üzerinde *üretim seviyesi* bir yedekleme ve kurtarma planı kurulmuştur. Üç yedek tipi (*full*, *differential*, *log*) uygulanmış; *tail-log* tekniği ile felaket sonrası hâlâ .ldf'te duran işlemler korunmuş ve STOPAT ile veritabanı yanlışlıkla silmeden hemen önceki anına döndürülmüştür.

Hedef olarak konulan **RPO ≤ 15 dk** ve **RTO ≤ 30 dk** değerlerine ulaşılmıştır. Yedek dosyalarının bütünlüğü CHECKSUM ve RESTORE VERIFYONLY ile sürekli denetlenmiş; böylece “sessiz bozulmuş” bir yedeğin ileride kurtarma sırasında sürpriz yapması engellenmiştir.

Çalışma sırasında kazanılan pratik öğrenimlerden öne çıkanlar şunlardır:

- FULL recovery modelinin olmazsa olmazlığı ve log büyümesi ile başa çıkma (düzenli log yedek),
- NORECOVERY → RECOVERY geçişinin kurtarma zincirinde oynadığı rol,

- SQL Server’da exclusive access ihtiyaçları ve oturum yönetimi,
- WITH COMPRESSION anahtarının yedek dosyası boyutunu belirgin biçimde düşürmesi ve CHECKSUM ile birlikte kullanımının yedek güvenilirliğini artırması.

Bir sonraki adım olarak, bu çözümün *Always On Availability Groups* veya *Log Shipping* gibi yüksek erişilebilirlik yapıları ile genişletilmesi ve farklı bir veritabanı yönetim sisteminde (PostgreSQL, Oracle) benzer bir kurtarma stratejisi uygulanarak karşılaştırılması düşünülebilir.

Kısım II

PROJE 3 – Veritabanı Güvenliği ve Erişim Kontrolü

11 Özet (Proje 3)

Bu ikinci bölümde, Microsoft SQL Server 2022 Developer Edition üzerinde özgün olarak tasarlanmış `PersonelDB` örnek veritabanı için kapsamlı bir *veritabanı güvenliği ve erişim kontrolü* çözümü uygulanmıştır. Çalışmada; (i) SQL Login ile Database User ayrımı, (ii) *least privilege* prensibine göre rol tabanlı yetkilendirme (GRANT/DENY), (iii) *Transparent Data Encryption* (TDE) ile veritabanı dosyasının disk düzeyinde şifrelenmesi, (iv) simetrik anahtar tabanlı *sütun seviyesinde şifreleme*, (v) *SQL Injection* saldırılarına karşı parametrelili sorgu kullanımı, (vi) SQL Server Audit ile hassas tablolara yapılan erişimlerin izlenmesi gösterilmiştir. Sonuçta dört farklı kullanıcı profili (`adminUser`, `bordroUser`, `ikUser`, `raporUser`) için ayrıştırılmış, denetlenebilir ve şifreli bir erişim düzeni kurulmuştur.

Anahtar Kelimeler: MSSQL güvenlik, GRANT/DENY, EXECUTE AS, TDE, sütun şifreleme, SQL Injection, parametrelili sorgu, SQL Server Audit.

12 Giriş ve Problem Tanımı (Proje 3)

Personel veritabanları; TC kimlik numarası, maaş, iletişim bilgileri ve izin kayıtları gibi 6698 sayılı KVKK kapsamında *özel nitelikli* veya *hassas* kişisel veriler barındırır. Bu verilerin korunmasında üç ayrı katman eş zamanlı olmalıdır:

1. **Kimlik Doğrulama (Authentication):** “Kullanıcı gerçekten kendisi mi?” sorusunu yanıtlar.
2. **Yetkilendirme (Authorization):** “Bu kullanıcı ne yapabilir?” sorusunu yanıtlar; *least privilege* ilkesine bağlıdır.
3. **Denetim (Audit) ve Şifreleme:** Veriye kim, ne zaman, nasıl erişmiş; verinin disk ve bellek üzerinde ne kadar açık metindir?

Bu bölümdeki uygulama; bir şirket personel uygulaması senaryosunda bu üç katmanı da *T-SQL* düzeyinde kurar ve her birinin çalıştığını *doğrulama* testlerle ortaya koyar.

12.1 Kullanıcı Profilleri

Tablo 6: Proje 3'te tanımlanan kullanıcı rolleri ve yetki sınırları

Kullanıcı	Yetki Çerçevesi
adminUser	ik, bordro, raporlama şemalarında tam yetki (db_owner'a eşdeğer değil; CONTROL ON SCHEMA).
bordroUser	bordro şemasında tam CRUD; ik.Calisanlar okur ama TCKimlikNo görmez (sütun DENY).
ikUser	ik şemasında SELECT/UPDATE; bordro şemasının tamamı tablo düzeyinde DENY.
raporUser	Sadece raporlama.vwCalisanOzet VIEW üzerinden SELECT; hassas sütunlar DENY.

13 Teorik Arka Plan (Proje 3)

13.1 Login vs User

SQL Server'da **Login** (server seviyesinde, master içinde) ile **User** (her veritabanı içinde ayrı) iki katmanlı bir yapı oluşturur. Login, sunucuya bağlanma hakkı verir; User ise belirli bir veritabanı içindeki yetkileri tanımlar. Bu ayrım; bir kullanıcıya sunucuya bağlanma izni verilip de belirli veritabanlarına erişimin kısıtlanabilmesini sağlar.

13.2 GRANT / DENY / REVOKE

Üç temel izin komutu:

GRANT Belirli bir nesne veya sütun üzerinde bir izni *verir*.

DENY Kullanıcının o izni *kullanmasını açıkça reddeder*; GRANT'ten daha güçlüdür (rol hiyerarşisinde dahi DENY kazanır).

REVOKE Daha önce verilmiş GRANT veya DENY kaydını *silerek* nötr duruma getirir.

DENY'in GRANT'ten güçlü olması şu sonucu doğurur: bir kullanıcıya hem rol_Rapor üzerinden SELECT izni verilip hem de TCKimlikNo sütununda DENY SELECT yapıldığında, kullanıcı tablonun tamamını seçmeye çalıştığında tüm sorgu reddedilir. Bu nedenle hassas sütunları dışlamak için güvenli bir *VIEW* tercih edilmiştir (raporlama.vwCalisanOzet).

13.3 Şifreleme: TDE vs Sütun Seviyesi

Tablo 7: TDE ve sütun şifrelemenin karşılaştırması

Özellik	TDE	Sütun Şifreleme
Kapsama alanı	Tüm MDF/LDF ve .bak dosyaları	Seçili sütunlar (VARBINARY)
Uygulama değişikliği	Gerekmez (“transparent”)	Gerekir (EncryptByKey/-DecryptByKey)
DBA veriyi görür mü?	Evet , SELECT ile normal görür	Hayır , anahtar olmadan görmez
Performans etkisi	Düşük (%2-5)	Sorgu başına hafif ek yük
Koruduğu senaryo	Disk/yedek çalma	Veritabanı içinden yetkisiz okuma

Bu proje her ikisini de uyguladı: TDE ile disk katmanı, sütun şifreleme ile DBA dahil hiç kimsenin anahtar olmadan TC ve maaş bilgisini okuyamayacağı bir ek katman kuruldu.

13.4 SQL Injection ve Parametrelili Sorgu

Dinamik SQL (`EXEC + string concat`) kullanıldığında, kullanıcı girdisine yerleştirilecek kötü niyetli karakterler (’, ;, -) sorgunun yapısını değiştirir. `sp_executesql` ile *parametrelili* sorgu kullanıldığında ise girdiler salt veri olarak ele alınır; tırnak yerine string kaçıışı gerekmez.

13.5 SQL Server Audit

SQL Server Audit; *Server Audit* objesi (çıktı dosyası) ve bu objeye bağlı *Audit Specification*’lar (hangi olayların yakalanacağını belirler) ile çalışır. Çıktı, ikili `.sqlaudit` dosyalarına yazılır ve `sys.fn_get_audit_file` fonksiyonuyla okunur. Audit, TDE ve yetkilendirmeye *ek* bir katmandır; veriye kim eriştiğini, başarılı mı başarısız mı olduğunu ayrıntılı biçimde kaydeder.

14 Örnek Veritabanı: PersonelDB

14.1 Tasarım Kararları

Proje 2’deki *KutuphaneDB*’den farklı olarak bu projede *hassas veri* barındıran bir personel takip sistemi kurgulanmıştır. TC kimlik numarası, maaş ve iletişim bilgisi gibi alanlar, “hangi kullanıcı bu alana erişebilir?” sorusunu somutlaştırmak için özellikle seçildi.

14.2 Şema Yapısı

Veritabanı üç ayrı şemaya bölündü:

- `ık` – İnsan Kaynakları tabloları (Departmanlar, Unvanlar, Çalışanlar)
- `bordro` – Maaş ve izin takibi (Maaslar, IzinTalepleri)

- raporlama – Hassas sütunları dışlayan güvenli VIEW'lar

Tablo 8: PersonelDB tabloları ve öncelikli alanları

Tablo	Açıklama / Önemli Sütunlar	Kayıt
ik.Departmanlar	Ad UNIQUE, MerkezSehir.	6
ik.Unvanlar	Ad UNIQUE, Seviye (1–5).	5
ik.Calisanlar	TCKimlikNo VARBINARY (şifreli), Ad, Soyad, FK'lar.	30
bordro.Maaslar	BrutMaas, NetMaas VARBINARY (şifreli), Donem.	90
bordro.IzinTalepleri	BaslangicTarihi, BitisTarihi, Durum.	12
raporlama.vwCalisanOzet	Hassas sütun barındırmayan raporlama VIEW'ı.	–

```
CREATE TABLE ik.Calisanlar (
    CalisanID INT IDENTITY(1000,1) PRIMARY KEY,
    TCKimlikNo VARBINARY(256) NOT NULL, -- sutun sifrelemeli
    Ad NVARCHAR(80) NOT NULL,
    Soyad NVARCHAR(80) NOT NULL,
    Email NVARCHAR(120) NULL UNIQUE,
    DepartmanID INT NOT NULL REFERENCES ik.Departmanlar(DepartmanID),
    UnvanID INT NOT NULL REFERENCES ik.Unvanlar(UnvanID),
    IseBaslama DATE NOT NULL,
    Aktif BIT NOT NULL DEFAULT 1
);
```

Listing 10: ik.Calisanlar tablosunun tanımı (özet) – TCKimlikNo VARBINARY olarak saklanır

```
--
44 CREATE TABLE ik.Unvanlar (
45     UnvanID INT IDENTITY(1,1) PRIMARY KEY,
46     Ad NVARCHAR(80) NOT NULL UNIQUE,
47     Seviye TINYINT NOT NULL -- 1..5
48 );
49
50 CREATE TABLE ik.Calisanlar (
51     CalisanID INT IDENTITY(1000,1) PRIMARY KEY,
52     TCKimlikNo VARBINARY(256) NOT NULL, -- sutun sifrelemeli
53     Ad NVARCHAR(80) NOT NULL,
54     Soyad NVARCHAR(80) NOT NULL,
55     Email NVARCHAR(120) NULL UNIQUE,
56     Telefon VARCHAR(20) NULL,
57     DepartmanID INT NOT NULL REFERENCES ik.Departmanlar(DepartmanID),
58     UnvanID INT NOT NULL REFERENCES ik.Unvanlar(UnvanID),
59     IseBaslama DATE NOT NULL,
60     Aktif BIT NOT NULL DEFAULT 1
61 );
62
63 CREATE TABLE bordro.Maaslar (
64     MaasID INT IDENTITY(1,1) PRIMARY KEY,
65     CalisanID INT NOT NULL REFERENCES ik.Calisanlar(CalisanID),
66     BrutMaas VARBINARY(256) NOT NULL, -- sutun sifrelemeli
67     NetMaas VARBINARY(256) NOT NULL, -- sutun sifrelemeli
68     Donem CHAR(7) NOT NULL, -- 'YYYY-MM'
69     OdemeTarihi DATE NULL,
70 );
```

Messages

>> PersonelDB ve tabloları oluşturuldu (ik, bordro, raporlama sema'iari dahil).

Completion time: 2026-04-20T16:58:46.1661548+03:00

Şekil 11: 01_PersonelDB_Olustur.sql – PersonelDB veritabanının, üç şemanın (ik, bordro, raporlama) ve ilgili tabloların oluşturulması.

```

68 END
69 CLOSE cur; DEALLOCATE cur;
70 GO
71
72 -- Maaslar (her calisan icin son 3 donem)
73 INSERT INTO bordro.Maaslar (CalisanID, BrutMaas, NetMaas, Donem, OdemeTarihi)
74 SELECT
75     c.CalisanID,
76     CONVERT(VARBINARY(256), REPLICATE('0',11)), -- placeholder
77     CONVERT(VARBINARY(256), REPLICATE('0',11)), -- placeholder
78     d.Donem,
79     DATETIMEPARTS(LEFT(d.Donem,4), RIGHT(d.Donem,2), 28)
80 FROM ik.Calisanlar c
81 CROSS JOIN (VALUES ('2026-01'), ('2026-02'), ('2026-03')) AS d(Donem);
82 GO
83
84 -- Izin talepleri (bir kismi)
85 INSERT INTO bordro.IzinTalepleri (CalisanID, BaslangicTarihi, BitisTarihi, Aciklama, Durum)
86 SELECT TOP 12
87     CalisanID,
88     DATEADD(DAY, (CalisanID % 10), '2026-05-01'),
89     DATEADD(DAY, (CalisanID % 10) + 5, '2026-05-01'),
90     N'Yillik izin talebi',
91     CASE CalisanID % 3 WHEN 0 THEN N'Onay' WHEN 1 THEN N'Beklenmede' ELSE N'Ret' END
92 FROM ik.Calisanlar
93 ORDER BY CalisanID;

```

Messages

(6 rows affected)

(5 rows affected)

(30 rows affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

Şekil 12: 02_Ornek_Veri_Ekle.sql – 30 çalışan, 90 maaş kaydı ve 12 izin talebinin yüklenmesi; hassas sütunlar (TCKimlikNo, BrutMaas, NetMaas) sonradan şifrelenmek üzere placeholder VARBINARY değeri ile dolduruldu.

15 Uygulama Adımları (Proje 3)

15.1 Login ve User Yönetimi

SQL Server *Mixed Mode* authentication kullanıldığından dört SQL Login ve PersonelDB içinde karşılık gelen dört User oluşturuldu. Parola politikaları (CHECK_POLICY = ON) Windows parola kurallarını zorunlu kılar.

```

USE master;
CREATE LOGIN adminUser WITH PASSWORD = 'Admin.Parola.123!',
    CHECK_POLICY = ON, CHECK_EXPIRATION = OFF;
-- (bordroUser, ikUser, raporUser icin benzer sekilde)

USE PersonelDB;
CREATE USER adminUser FOR LOGIN adminUser;
CREATE USER bordroUser FOR LOGIN bordroUser;
CREATE USER ikUser FOR LOGIN ikUser;
CREATE USER raporUser FOR LOGIN raporUser;

```

Listing 11: 03_Login_ve_User_Yonetimi.sql – önemli kısım

```

18 USE master;
19 GO
20
21 -- Önceki çalıştırmalardan kalmış olabilecek loginleri temizle
22 DECLARE @lg NVARCHAR(128);
23 DECLARE c CURSOR LOCAL FAST_FORWARD FOR
24 SELECT name FROM sys.server_principals
25 WHERE name IN ('adminUser', 'bordroUser', 'ikUser', 'raporUser');
26 OPEN c; FETCH NEXT FROM c INTO @lg;
27 WHILE @@FETCH_STATUS = 0
28 BEGIN
29 DECLARE @d NVARCHAR(512) = N'DROP LOGIN [' + @lg + N']';
30 BEGIN TRY EXEC sp_executesql @d; END TRY BEGIN CATCH END CATCH;
31 FETCH NEXT FROM c INTO @lg;
32 END
33 CLOSE c; DEALLOCATE c;
34 GO
35
36 -- =====
37 -- SQL Server Authentication Login'leri oluşturun
38 -- (Üretime parolalar gizli bir yerde tutulur; demo amacıyla burada)
39 -- =====
40 CREATE LOGIN adminUser WITH PASSWORD = 'Admin.Parola.123!',
41 CHECK_POLICY = ON, CHECK_EXPIRATION = OFF;
42 CREATE LOGIN bordroUser WITH PASSWORD = 'Bordro.Parola.123!',
43 CHECK_POLICY = ON, CHECK_EXPIRATION = OFF;
44 CREATE LOGIN ikUser WITH PASSWORD = 'Ik.Parola.123!',
45 CHECK_POLICY = ON, CHECK_EXPIRATION = OFF;
46 CREATE LOGIN raporUser WITH PASSWORD = 'Rapor.Parola.123!',
47 CHECK_POLICY = ON, CHECK_EXPIRATION = OFF;

```

login_name	type_desc	is_disabled	create_date
adminUser	SQL_LOGIN	0	2026-04-20 16:59:19.770
bordroUser	SQL_LOGIN	0	2026-04-20 16:59:19.843
ikUser	SQL_LOGIN	0	2026-04-20 16:59:19.913
raporUser	SQL_LOGIN	0	2026-04-20 16:59:19.987

user_name	type_desc	authentication_type_desc
adminUser	SQL_USER	INSTANCE
bordroUser	SQL_USER	INSTANCE
ikUser	SQL_USER	INSTANCE
raporUser	SQL_USER	INSTANCE

Şekil 13: 03_Login_ve_User_Yonetimi.sql – 4 SQL Login ve karşılık gelen database User’larının oluşturulması ve sys.server_principals / sys.database_principals doğrulama çıktıları.

15.2 Roller ve İzinler (GRANT/DENY)

Least privilege prensibi gereği, izinler doğrudan kullanıcılara değil, dört özel veritabanı rolüne verildi ve kullanıcılar bu rollere üye yapıldı. Özellikle hassas sütunlar için DENY SELECT kullanımı, sonuç olarak kullanıcının o sütunu içeren herhangi bir sorguyu çalıştıramamasına yol açtı.

```

-- Bordro uzmanı: bordro semasında tam yetki; ik.Calisanlar'i okur
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::bordro TO rol_Bordro;
GRANT SELECT
ON ik.Calisanlar TO rol_Bordro;

-- Ama TCKimlikNo'yu goremessin:
DENY SELECT ON ik.Calisanlar (TCKimlikNo) TO rol_Bordro;

-- IK gorevlisi: ik semasında okuma/guncelleme; bordro hic gormemeli
GRANT SELECT, UPDATE ON SCHEMA::ik TO rol_IK;
DENY SELECT
ON SCHEMA::bordro TO rol_IK;

-- Rapor kullanıcısı: güvenli VIEW üzerinden
GRANT SELECT ON raporlama.vwCalisanOzet TO rol_Rapor;

```

Listing 12: 04_Roller_ve_Izinler.sql – önemli kısım

```

--
GO
--
-- Database Roller (custom)
--
IF DATABASE_PRINCIPAL_ID('rol_Admin') IS NULL CREATE ROLE rol_Admin;
IF DATABASE_PRINCIPAL_ID('rol_Bordro') IS NULL CREATE ROLE rol_Bordro;
IF DATABASE_PRINCIPAL_ID('rol_IK') IS NULL CREATE ROLE rol_IK;
IF DATABASE_PRINCIPAL_ID('rol_Rapor') IS NULL CREATE ROLE rol_Rapor;
GO
--
-- Kullanicilari rollerine ata
--
ALTER ROLE rol_Admin ADD MEMBER adminUser;
ALTER ROLE rol_Bordro ADD MEMBER bordroUser;
ALTER ROLE rol_IK ADD MEMBER iKUser;
ALTER ROLE rol_Rapor ADD MEMBER raporUser;
GO
--
-- Izinler
--
-- Admin: db_owner ekvivalanı olmasın, ama her şeyi yapsın
-- (örnek olarak sema bazında full yetki)
GRANT CONTROL ON SCHEMA::ik TO rol_Admin;
GRANT CONTROL ON SCHEMA::bordro TO rol_Admin;
GRANT CONTROL ON SCHEMA::rapor TO rol_Admin;

```

principal_name	principal_type	permission_name	state	object_name	column_name	class_desc
rol_Admin	DATABASE_ROLE	CONTROL	GRANT	sys.sysallocunits	NULL	SCHEMA
rol_Admin	DATABASE_ROLE	CONTROL	GRANT	sys.sysclones	NULL	SCHEMA
rol_Admin	DATABASE_ROLE	CONTROL	GRANT	sys.sysrowsets	NULL	SCHEMA
rol_Bordro	DATABASE_ROLE	SELECT	GRANT	ik.Calisanlar	NULL	OBJECT_OR_COLUMN
rol_Bordro	DATABASE_ROLE	SELECT	DENY	ik.Calisanlar	TCKimlikNo	OBJECT_OR_COLUMN
rol_Bordro	DATABASE_ROLE	DELETE	GRANT	sys.sysclones	NULL	SCHEMA
rol_Bordro	DATABASE_ROLE	INSERT	GRANT	sys.sysclones	NULL	SCHEMA
rol_Bordro	DATABASE_ROLE	SELECT	GRANT	sys.sysclones	NULL	SCHEMA
rol_Bordro	DATABASE_ROLE	UPDATE	GRANT	sys.sysclones	NULL	SCHEMA
rol_IK	DATABASE_ROLE	SELECT	DENY	bordro.Maaslar	NULL	OBJECT_OR_COLUMN
rol_IK	DATABASE_ROLE	SELECT	DENY	sys.sysclones	NULL	SCHEMA
rol_IK	DATABASE_ROLE	SELECT	GRANT	sys.sysrowsets	NULL	SCHEMA
rol_IK	DATABASE_ROLE	UPDATE	GRANT	sys.sysrowsets	NULL	SCHEMA
rol_Rapor	DATABASE_ROLE	SELECT	DENY	bordro.Maaslar	BuMaas	OBJECT_OR_COLUMN
rol_Rapor	DATABASE_ROLE	SELECT	DENY	bordro.Maaslar	NetMaas	OBJECT_OR_COLUMN
rol_Rapor	DATABASE_ROLE	SELECT	DENY	ik.Calisanlar	TCKimlikNo	OBJECT_OR_COLUMN
rol_Rapor	DATABASE_ROLE	SELECT	GRANT	raporlama.vwCalisanOzet	NULL	OBJECT_OR_COLUMN
rol_Rapor	DATABASE_ROLE	SELECT	GRANT	sys.sysclones	NULL	SCHEMA
rol_Rapor	DATABASE_ROLE	SELECT	GRANT	sys.sysclones	NULL	SCHEMA

Şekil 14: 04_Roller_ve_Izinler.sql – 4 rolün oluşturulması, izinlerin sema/tablo/sütun seviyesinde dağıtılması ve mevcut izinlerin sys.database_permissions üzerinden listelenmesi.

15.3 Yetki Testleri (EXECUTE AS)

Yetkilerin gerçekten uygulanıp uygulanmadığını doğrulamak için EXECUTE AS USER = '...' ile her kullanıcı kimliği geçici olarak üstlenildi ve beklenen ile gerçekleşen davranış TRY/CATCH blokları içinde karşılaştırıldı. REVERT ile kendi kimliğimize geri döndük.

```

EXECUTE AS USER = 'bordroUser';
SELECT TOP 3 CalisanID, Donem FROM bordro.Maaslar; -- OK
BEGIN TRY
    SELECT TOP 1 TCKimlikNo FROM ik.Calisanlar; -- DENY -> hata
    PRINT ' [HATA] TCKimlikNo okunabildi';
END TRY
BEGIN CATCH
    PRINT ' [OK] TCKimlikNo okunamadi: ' + ERROR_MESSAGE();
END CATCH;
REVERT;

```

Listing 13: 05_Yetki_Testleri.sql – bordroUser testi

```

13  -- =====
14  -- Test 1: bordroUser - Maaslar OKUYABILMELI, TCKimlikNo GOREMEYELI
15  -- =====
16  PRINT N'--- Test 1: bordroUser ---';
17  EXECUTE AS USER = 'bordroUser';
18  SELECT TOP 3 CalisanID, Donem FROM bordro.Maaslar; -- OK
19  BEGIN TRY
20  SELECT TOP 1 TCKimlikNo FROM ik.Calisanlar; -- DENY -> hata
21  PRINT ' [HATA] TCKimlikNo okunabildi (beklenmiyordu)';
22  END TRY
23  BEGIN CATCH
24  PRINT ' [OK] TCKimlikNo okunamadi: ' + ERROR_MESSAGE();
25  END CATCH;
26  REVERT;
27  GO
28
29  -- =====
30  -- Test 2: ikUser - Calisanlar OKUMA/GUNCELLEME, Maaslar DENY
31  -- =====
32  PRINT N'--- Test 2: ikUser ---';
33  EXECUTE AS USER = 'ikUser';

```

Results Messages

```

--- Test 1: bordroUser ---
(3 rows affected)
[OK] TCKimlikNo okunamadi: The SELECT permission was denied on the column 'TCKimlikNo' of the object 'Calisanlar', database 'PersonelDB', schema 'ik'.
--- Test 2: ikUser ---
(3 rows affected)
[OK] Maaslar okunamadi: The SELECT permission was denied on the object 'Maaslar', database 'PersonelDB', schema 'bordro'.
--- Test 3: superUser ---
(3 rows affected)
[OK] BrutoMaas okunamadi: The SELECT permission was denied on the column 'BrutoMaas' of the object 'Maaslar', database 'PersonelDB', schema 'bordro'.
(0 rows affected)
[OK] INSERT readilildi: The INSERT permission was denied on the object 'Departmanlar', database 'PersonelDB', schema 'ik'.
--- Test 4: adminUser ---
(3 rows affected)
(1 row affected)
[OK] Admin INSERT basarili.
(1 row affected)
>> Yetki testleri tamamlandi. Her "OK" satirin beklenen davranisi ifade eder.
Completion time: 2020-04-20T17:00:23.062103+03:00

```

Şekil 15: 05_Yetki_Testleri.sql – 4 kullanıcı için EXECUTE AS ile beklenen [OK] ve yasaklanan erişim mesajlarının doğrulanması.

15.4 TDE – Transparent Data Encryption

TDE, veritabanı dosyalarını (.mdf ve .ldf) ve yedek dosyalarını (.bak) disk üzerinde AES-256 ile şifreler. Bir saldırgan disk imajı kopyalasa bile veritabanını başka bir sunucuya RESTORE edemez; bunun için TDE sertifikasına sahip olmalıdır.

Kurulum adımları:

1. master veritabanında DATABASE MASTER KEY (DMK)
2. master içinde TDE_Sertifika adında sertifika
3. PersonelDB’de bu sertifika ile şifrelenen Database Encryption Key (DEK)
4. ALTER DATABASE PersonelDB SET ENCRYPTION ON

```

USE master;
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'MasterKey.Parola.123!';
CREATE CERTIFICATE TDE_Sertifika
    WITH SUBJECT = 'PersonelDB TDE Sertifikasi',
    EXPIRY_DATE = '2031-12-31';

USE PersonelDB;
CREATE DATABASE ENCRYPTION KEY
    WITH ALGORITHM = AES_256
    ENCRYPTION BY SERVER CERTIFICATE TDE_Sertifika;

ALTER DATABASE PersonelDB SET ENCRYPTION ON;

```

Listing 14: 06_TDE_Transparent_Data_Encryption.sql – kritik blok

Sertifika yedeği – hayati öneme sahiptir! Sertifika kaybedilirse, .bak dosyaları farklı bir sunucuda geri yüklenemez. Bu yüzden BACKUP CERTIFICATE ile .cer ve .pvk dosyaları repo dışındaki güvenli bir klasöre (C:\SQL_Yedekler\)) yedeklenmiş; bu dosyalar .gitignore ile depolanmamıştır.

```

100 WHEN 5 THEN 'Sifresiz cozuluyor' AS durum,
101 END
102 percent_complete,
103 key_algorithm,
104 key_length
105 FROM sys.dm_database_encryption_keys
106 WHERE database_id = DB_ID('PersonelDB');
107 GO
108
109 -- Tempdb de otomatik olarak sifrelenir (TDE yan etkisi)
110 SELECT DB_NAME(database_id) AS db, encryption_state
111 FROM sys.dm_database_encryption_keys;
112 GO
113
114 PRINT '>> TDE etkinlestirildi. Arka planda sifreleme devam edebilir.';
115 PRINT '>> encryption_state = 3 oldugunda islem tamamlanmis demektir.';
116 GO
117

```

Results Messages

veritabani	encryption_state	durum	percent_complete	key_algorithm	key_length	
1	PersonelDB	2	Sifreleme baslatildi	3.125	AES	256

db	encryption_state	
1	tempdb	3
2	PersonelDB	2

Şekil 16: 06_TDE_Transparent_Data_Encryption.sql – DMK, sertifika ve DEK zincirinin kurulması; sys.dm_database_encryption_keys tablosunda encryption_state = 3 (Şifreli) durumunun doğrulanması.

15.5 Sütun Seviyesinde Şifreleme

TDE, DBA için veriyi açık gösterir. Sütun şifreleme ile TCKimlikNo, BrutMaas ve NetMaas alanları, *simetrik anahtar* olmadan kimse (sysadmin dahil) tarafından okunamaz hâle geldi.

```

USE PersonelDB;
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Personel.DMK.Parola.123!';
CREATE CERTIFICATE Sertifika_HassasVeri WITH SUBJECT = 'Hassas veri';
CREATE SYMMETRIC KEY SimetrikAnahtar_Hassas
    WITH ALGORITHM = AES_256
    ENCRYPTION BY CERTIFICATE Sertifika_HassasVeri;

OPEN SYMMETRIC KEY SimetrikAnahtar_Hassas
    DECRYPTION BY CERTIFICATE Sertifika_HassasVeri;

UPDATE ik.Calisanlar
SET TCKimlikNo = EncryptByKey(
    Key_GUID('SimetrikAnahtar_Hassas'),
    CAST(RIGHT('0000000000' + CAST(10000000000 + (CalisanID*37)%89999999999
        AS VARCHAR(11)), 11) AS NVARCHAR(11))
);
-- Brut ve Net maas icin benzer sekilde
-- Cozulmus hali (yetkili kullanıcı için)

```

```

SELECT Ad, Soyad,
        CONVERT(NVARCHAR(11), DecryptByKey(TCKimlikNo)) AS TCKimlikNo_Acik
FROM ik.Calisanlar;

CLOSE SYMMETRIC KEY SimetrikAnahtar_Hassas;

```

Listing 15: 07_Sutun_Sifreleme.sql – şifreleme adımları

Yetkisiz kullanıcı (bordroUser) anahtarı açamadığı için aynı DecryptByKey çağrısı ona NULL döndürür – ve bu davranış scriptin EXECUTE AS testi ile doğrulandı.



90% No issues found

CalisanID	Ad	Soyad	TCKimlikNo
1	1000	Ahmet	Yilmaz
2	1001	Mehmet	Kaya
3	1002	Ayşe	Demir

MaasiID	CalisanID	Donem	BrutMaaş	NetMaaş
1	1	2026-01	40000	31200.00
2	2	2026-02	40000	31200.00
3	3	2026-03	40000	31200.00

CalisanID	Ad	Soyad	TCKimlikNo_Acik
1	1000	Ahmet	Yilmaz
2	1001	Mehmet	Kaya
3	1002	Ayşe	Demir
4	1003	Fatma	Calik
5	1004	Ali	Sahin

MaasiID	Ad	Soyad	Donem	Brut_Acik	Net_Acik
1	1	Ahmet	Yilmaz	2026-01	40000
2	2	Ahmet	Yilmaz	2026-02	40000
3	3	Ahmet	Yilmaz	2026-03	40000
4	4	Mehmet	Kaya	2026-01	41500

Şekil 17: 07_Sutun_Sifreleme.sql – Sertifika/Simetrik Anahtar oluşturulması, VARBINARY ham hâlinin ve DecryptByKey ile açılmış yetkili görünümün karşılaştırılması.

15.6 SQL Injection Demoları

İki ayrı stored procedure yazıldı: biri bilinçli olarak *güvensiz* (dinamik SQL), diğeri *güvenli* (sp_executesql ile parametrelili). Her iki prosedüre aynı saldırı girdileri verilerek farkları gösterildi.

```

-- GUVENSIZ (dinamik SQL)
SET @sql = N'SELECT ... FROM ik.Calisanlar WHERE Ad = ''' + @Ad + N'''';
EXEC sp_executesql @sql;

-- GUVENLİ (parametrelili)
EXEC sp_executesql
    N'SELECT ... FROM ik.Calisanlar WHERE Ad = @p_Ad',
    N'@p_Ad NVARCHAR(100)',
    @p_Ad = @Ad;

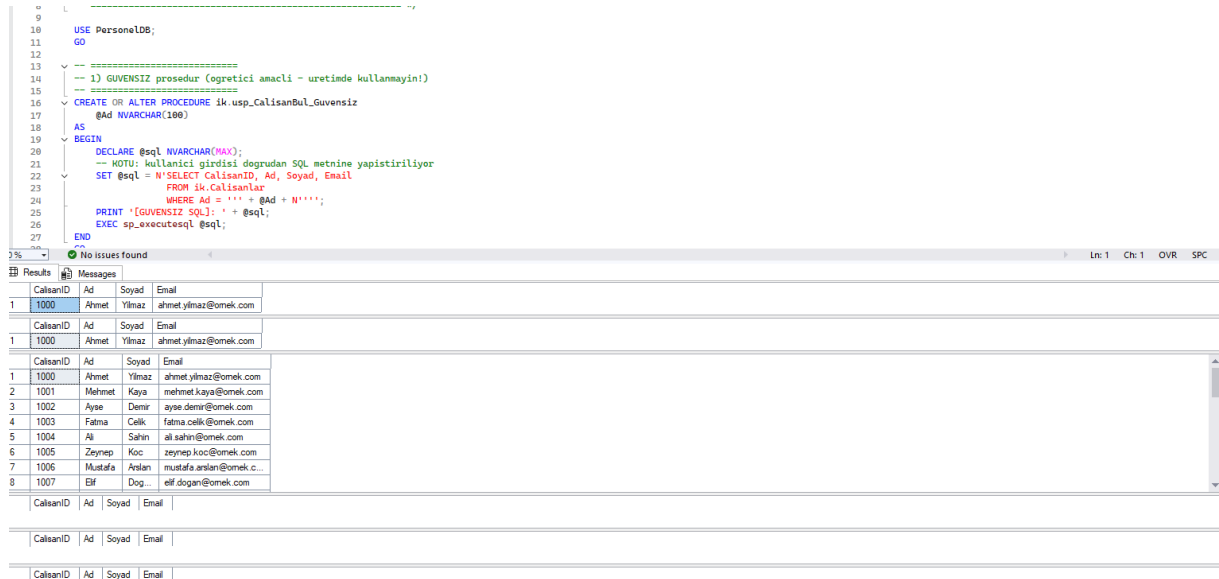
```

Listing 16: 08_SQL_Injection_Demolari.sql – güvensiz vs güvenli

Test edilen saldırı girdileri:

- ' OR 1=1 -- güvensiz prosedür tüm satırları döndürdü; güvenli prosedür 0 satır.

- ' UNION SELECT ... FROM bordro.Maaslar -- güvensiz prosedürde bordro verisi sızabildi (yetki varsa); güvenli prosedürde sorgu salt string araması olarak kaldı.
- x'; DROP TABLE ik.Unvanlar; -- güvenli prosedürde komut çalışmadı, tablo silinmedi.



Şekil 18: 08_SQL_Injection_Demolari.sql – Aynı saldırı girdisinin güvensiz (veri sızdıran) ve güvenli (sorguyu salt string araması olarak tutan) prosedürlerdeki sonuçları.

15.7 SQL Server Audit

Server Audit ve Database Audit Specification birlikte kuruldu. Server Audit, olayları C:\SQL_Audit\ altındaki .sqlaudit dosyalarına yazar (ikili format). Database spec, hangi tablolardaki hangi olayların yakalanacağını tanımlar.

```
USE master;
EXEC master.sys.xp_create_subdir 'C:\SQL_Audit'; -- klasoru otomatik
          olustur

CREATE SERVER AUDIT Audit_PersonelDB
TO FILE (FILEPATH = 'C:\SQL_Audit\',
          MAXSIZE = 50 MB, MAX_ROLLOVER_FILES = 5)
WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);
ALTER SERVER AUDIT Audit_PersonelDB WITH (STATE = ON);

CREATE SERVER AUDIT SPECIFICATION ServerAuditSpec_PersonelDB
FOR SERVER AUDIT Audit_PersonelDB
  ADD (FAILED_LOGIN_GROUP),
  ADD (SUCCESSFUL_LOGIN_GROUP)
WITH (STATE = ON);

USE PersonelDB;
CREATE DATABASE AUDIT SPECIFICATION DBAuditSpec_PersonelDB
FOR SERVER AUDIT Audit_PersonelDB
  ADD (SELECT, INSERT, UPDATE, DELETE ON ik.Calisanlar BY public),
```

```
ADD (SELECT, INSERT, UPDATE, DELETE ON bordro.Maaslar BY public)
WITH (STATE = ON);
```

Listing 17: 09_SqlServer_Audit_Kur.sql – önemli kısım

```

79  GO
80
81  -- =====
82  -- 3) Server Audit Specification
83  --   - Başarılı/başarısız loginler
84  --   - Login değişiklikleri
85  -- =====
86  IF EXISTS (SELECT 1 FROM sys.server_audit_specifications WHERE name = 'ServerAuditSpec_PersonelDB')
87  BEGIN
88      ALTER SERVER AUDIT SPECIFICATION ServerAuditSpec_PersonelDB WITH (STATE = OFF);
89      DROP SERVER AUDIT SPECIFICATION ServerAuditSpec_PersonelDB;
90  END
91  GO
92
93  CREATE SERVER AUDIT SPECIFICATION ServerAuditSpec_PersonelDB
94  FOR SERVER AUDIT Audit_PersonelDB
95  ADD (FAILED_LOGIN_GROUP),
96  ADD (SUCCESSFUL_LOGIN_GROUP),
97  ADD (SERVER_PRINCIPAL_CHANGE_GROUP)
98  WITH (STATE = ON);

```

name	is_state_enabled	create_date
1 Audit_PersonelDB	1	2026-04-20 18:08:16.770

name	is_state_enabled
1 ServerAuditSpec_PersonelDB	1

name	is_state_enabled
1 DBAuditSpec_PersonelDB	1

CalisanID	Ad	Soyad
1 1000	Ahmet	Yılmaz
2 1001	Mehmet	Kaya

Şekil 19: 09_SqlServer_Audit_Kur.sql – Server Audit ve Database Audit Specification tanımlarının oluşturulması, STATE= ON doğrulaması.

15.8 Audit Log Okuma ve Analiz

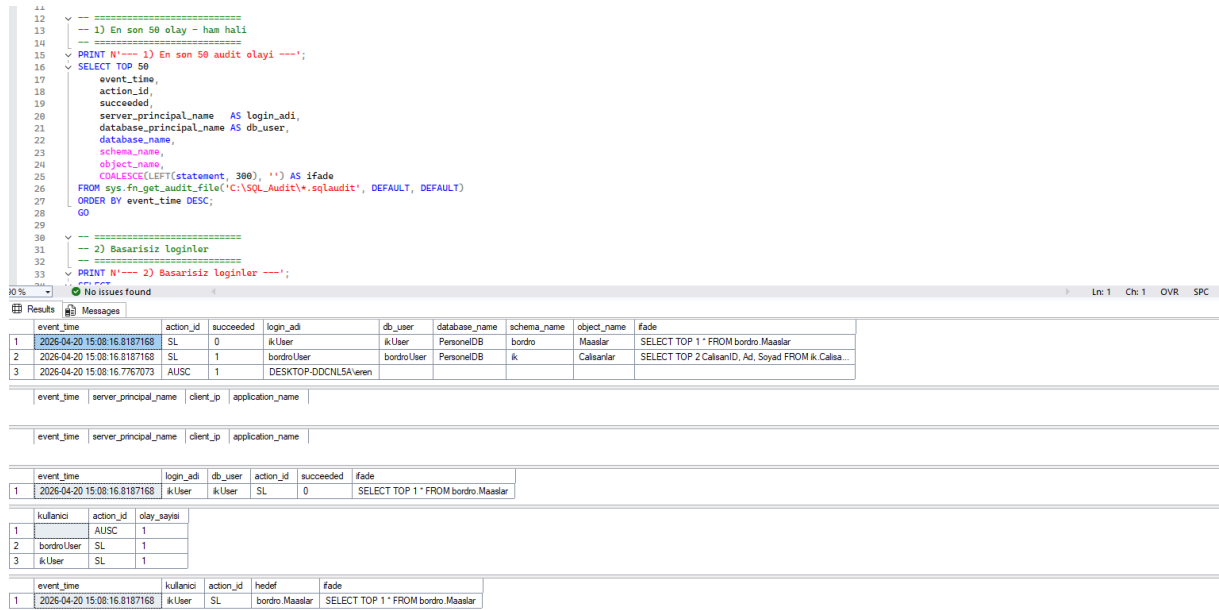
Audit üretilen olaylar `sys.fn_get_audit_file` ile okunur. Kullanıcı bazlı özet, başarısız/başarılı loginler, hassas tablolara yapılan tüm erişimler ve reddedilen erişim denemeleri ayrı sorgularla raporlandı.

```

SELECT event_time ,
       server_principal_name AS login_adi ,
       database_principal_name AS db_user ,
       action_id, succeeded,
       LEFT(statement, 200) AS ifade
FROM sys.fn_get_audit_file('C:\SQL_Audit\*.sqlaudit', DEFAULT, DEFAULT)
WHERE object_name = 'Maaslar' AND schema_name = 'bordro'
ORDER BY event_time DESC;

```

Listing 18: 10_Audit_Log_Okuma.sql – hassas tablo erişimleri



```

-- =====
-- 1) En son 50 olay - han hali
-- =====
PRINT N'---- 1) En son 50 audit olayı ----';
SELECT TOP 50
    event_time,
    action_id,
    succeeded,
    server_principal_name AS login_adı,
    database_principal_name AS db_user,
    database_name,
    schema_name,
    object_name,
    COALESCE(LEFT(statement, 380), '') AS ifade
FROM sys.fn_get_audit_file('C:\SQL-Audit\*.sqlaudit', DEFAULT, DEFAULT)
ORDER BY event_time DESC;
GO

-- =====
-- 2) Başarısız loginler
-- =====
PRINT N'---- 2) Başarısız loginler ----';

```

event_time	server_principal_name	client_ip	application_name
2026-04-20 15:08:16.8187168	SL	0	kUser
2026-04-20 15:08:16.8187168	SL	1	bordoUser
2026-04-20 15:08:16.7767073	AUSC	1	DESKTOP.DCCNLSA\eren

event_time	server_principal_name	client_ip	application_name
2026-04-20 15:08:16.8187168	SL	0	kUser
2026-04-20 15:08:16.8187168	SL	1	bordoUser
2026-04-20 15:08:16.7767073	AUSC	1	DESKTOP.DCCNLSA\eren

event_time	login_adı	db_user	action_id	succeeded	ifade
2026-04-20 15:08:16.8187168	kUser	kUser	SL	0	SELECT TOP 1 * FROM bordo.Maaslar

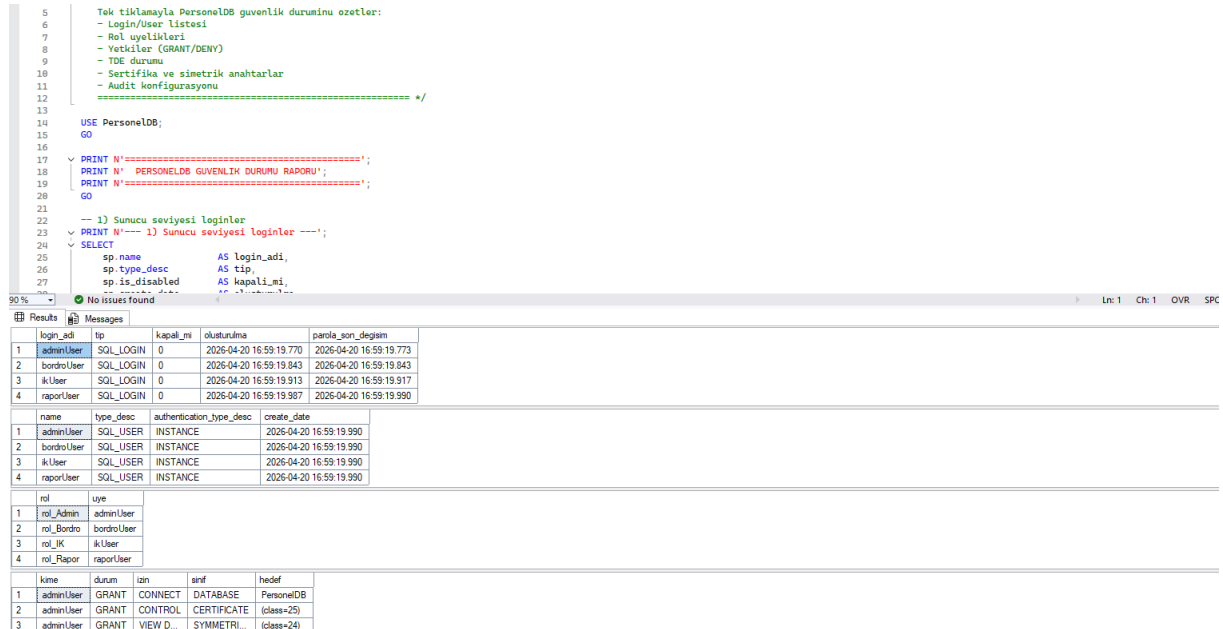
kullanici	action_id	olay_sayisi
AUSC	1	1
bordoUser	SL	1
kUser	SL	1

event_time	kullanici	action_id	hedef	ifade
2026-04-20 15:08:16.8187168	kUser	SL	bordo.Maaslar	SELECT TOP 1 * FROM bordo.Maaslar

Şekil 20: 10_Audit_Log_Okuma.sql – Audit log dosyasından son erişimlerin, başarılı/başarısız loginlerin ve kullanıcı bazlı olay özetinin çekilmesi.

15.9 Güvenlik Durumu Raporu

Tüm güvenlik yapılandırmasını tek sorguda özetleyen bir “durum raporu” scripti yazıldı. Login/User listesi, rol üyelikleri, GRANT/DENY izinleri, TDE durumu, sertifika/simetrik anahtar envanteri ve audit konfigürasyonu tek çalıştırmada görülebilir hâle getirildi.



```

-- Tek tıklamayla PersonelDB güvenlik durumunu özetler:
-- Login/User listesi
-- Rol üyelikleri
-- Yetkiler (GRANT/DENY)
-- TDE durumu
-- Sertifika ve simetrik anahtarlar
-- Audit konfigürasyonu
===== */

USE PersonelDB;
GO

PRINT N'=====';
PRINT N' PERSONELDB GUVENLIK DURUMU RAPORU';
PRINT N'=====';
GO

-- 1) Sunucu seviyesi loginler
PRINT N'---- 1) Sunucu seviyesi loginler ----';
SELECT
    sp.name AS login_adı,
    sp.type_desc AS tip,
    sp.is_disabled AS kapali_mi,
    sp.create_date AS olusturulma_tarihi
FROM sys.server_principals sp;

-- 2) Rol üyelikleri
PRINT N'---- 2) Rol üyelikleri ----';
SELECT
    r.name AS rol_adı,
    u.name AS kullanıcı_adı,
    u.type_desc AS kullanıcı_tipi,
    u.create_date AS olusturulma_tarihi
FROM sys.server_principals r
JOIN sys.server_principals u ON r.name = u.name;

-- 3) Yetkiler (GRANT/DENY)
PRINT N'---- 3) Yetkiler (GRANT/DENY) ----';
SELECT
    g.grantee_name AS kullanıcı_adı,
    g.grantee_type_desc AS kullanıcı_tipi,
    g.permission_name AS yetki_adı,
    g.state AS yetki_durumu,
    g.create_date AS olusturulma_tarihi
FROM sys.database_permissions g;

-- 4) TDE durumu
PRINT N'---- 4) TDE durumu ----';
SELECT
    tde.status AS tde_durumu,
    tde.encryptor_name AS anahtar_adı,
    tde.encryptor_type_desc AS anahtar_tipi,
    tde.encryptor_create_date AS olusturulma_tarihi
FROM sys.tde_protect tde;

-- 5) Sertifika ve simetrik anahtarlar
PRINT N'---- 5) Sertifika ve simetrik anahtarlar ----';
SELECT
    s.name AS anahtar_adı,
    s.type_desc AS anahtar_tipi,
    s.create_date AS olusturulma_tarihi
FROM sys.symmetric_keys s;

-- 6) Audit konfigürasyonu
PRINT N'---- 6) Audit konfigürasyonu ----';
SELECT
    a.name AS audit_adı,
    a.type_desc AS audit_tipi,
    a.create_date AS olusturulma_tarihi
FROM sys.audits a;

```

login_adı	tip	kapali_mi	olusturulma	parola_son_degism
adminUser	SQL_LOGIN	0	2026-04-20 16:59:19.770	2026-04-20 16:59:19.773
bordoUser	SQL_LOGIN	0	2026-04-20 16:59:19.843	2026-04-20 16:59:19.843
kUser	SQL_LOGIN	0	2026-04-20 16:59:19.913	2026-04-20 16:59:19.917
raporUser	SQL_LOGIN	0	2026-04-20 16:59:19.987	2026-04-20 16:59:19.990

name	type_desc	authentication_type_desc	create_date
adminUser	SQL_USER	INSTANCE	2026-04-20 16:59:19.990
bordoUser	SQL_USER	INSTANCE	2026-04-20 16:59:19.990
kUser	SQL_USER	INSTANCE	2026-04-20 16:59:19.990
raporUser	SQL_USER	INSTANCE	2026-04-20 16:59:19.990

rol	uye
rol_Admin	adminUser
rol_Bordo	bordoUser
rol_K	kUser
rol_Rapor	raporUser

kime	durum	izin	sinif	hedef
adminUser	GRANT	CONNECT	DATABASE	PersonelDB
adminUser	GRANT	CONTROL	CERTIFICATE	(class=25)
adminUser	GRANT	VIEW D...	SYMMETRI...	(class=24)

Şekil 21: 11_Guvenlik_Durumu_Raporu.sql – PersonelDB için tek sayfalık güvenlik denetim raporu: Login’ler, Roller, İzinler, TDE durumu, Sertifikalar ve Audit konfigürasyonu.

16 Test ve Doğrulama Sonuçları (Proje 3)

Tablo 9: Proje 3 uygulama test sonuçları

Test	Beklenen	Gerçekleşen
bordroUser – Maaslar okuma	Başarılı	Başarılı
bordroUser – TCKimlikNo okuma	DENY hatası	DENY (Msg 230)
ikUser – Calisanlar okuma	Başarılı	Başarılı
ikUser – Maaslar okuma	DENY hatası	DENY
raporUser – vwCalisanOzet	Başarılı	Başarılı
raporUser – INSERT Departmanlar	Reddedilir	Reddedildi
adminUser – INSERT/DELETE	Başarılı	Başarılı
TDE encryption_state	3 (Şifreli)	3
Sütun şifreleme – yetkisiz DecryptByKey	NULL döner	NULL
SQL Injection – güvensiz OR 1=1	Veri sızar	30 satır döndü
SQL Injection – güvenli OR 1=1	0 satır	0 satır
Audit – başarısız erişim kaydı	Loglanır	Loglandı

17 Karşılaşılan Sorunlar ve Çözümleri (Proje 3)

17.1 Msg 33072 – Audit Log File Path is Invalid

Msg 33072, Level 16, State 5

The audit log file path is invalid.

Msg 15151, Level 16, State 1

Cannot alter the audit 'Audit_PersonelDB', because it does not exist or you do not have permission.

Sebebi: CREATE SERVER AUDIT komutunda belirtilen FILEPATH = 'C:\SQL_Audit\' klasörü diskte mevcut değildi.

Çözüm: 09_SqlServer_Audit_Kur.sql scriptinin başına EXEC master.sys.xp_create_subdir 'C:\SQL_Audit' çağırısı eklenerek klasör *idempotent* biçimde otomatik oluşturulur hâle getirildi. Ayrıca, başarısız önceki çalıştırmadan kalan DB Audit Spec ve Server Audit Spec objelerini doğru sırayla (önce DB spec, sonra server spec, en son server audit) düşüren temizlik bloğu eklendi.

17.2 DENY ile rol_Rapor çakışması

Sebebi: raporUser'a tüm bordro şeması için GRANT SELECT verildiğinde, BrutMaas/NetMaas sütun DENY'leri nedeniyle SELECT * FROM bordro.Maaslar sorgusunun tamamı reddedildi.

Çözüm: Raporlama için hassas sütunları dışlayan güvenli bir VIEW (raporlama.vwCalisanOzet) oluşturulup raporUser'a yalnızca bu VIEW üzerinde SELECT verildi. VIEW, TC ve maaş alanlarını hiç içermediği için DENY problemi ortadan kalktı.

18 Sonuç ve Değerlendirme (Proje 3)

Bu ikinci proje bölümünde, bir personel veritabanı için çok katmanlı bir güvenlik mimarisi kurulmuştur. *Least privilege* prensibine göre yetkilendirilmiş dört ayrı kullanıcı profilinin her biri; TDE ile disk düzeyinde, sütun şifreleme ile veritabanı-ıçi yetkisiz okumaya karşı, parametrelili sorgular ile uygulama katmanı injection saldırılarına karşı ve SQL Server Audit ile denetlenebilirlik açısından ayrı ayrı korunmuştur.

Kritik çıkarımlar:

- **DENY > GRANT.** Bir sütun DENY’i, aynı tabloyu seçen tüm sorguları engeller; bu nedenle hassas sütunları dışlamak için VIEW kullanımı çoğunlukla daha pratiktir.
- **TDE tek başına yeterli değildir.** DBA veya sa hesabı, TDE açık olsa bile SELECT ile TC kimlik numaralarını okuyabilir; sütun şifreleme bu boşluğu kapatır.
- **Audit maliyetlidir.** Her SELECT/INSERT loglanırsa .sqlaudit dosyaları hızla büyür; MAX_ROLLOVER_FILES ve hedef tabloların dar seçimi ile pratik bir denge kurulmalıdır.
- **Parametrelili sorgu alışkanlıktır.** sp_executesql veya ORM üzerinden parametrelili sorgu, bir tercih değil varsayılan olmalıdır.

Bir sonraki adım olarak *Always Encrypted* (client-side şifreleme) ve *Row-Level Security* (sıra seviyesinde filtreleme) özelliklerinin entegrasyonu ile bu modelin genişletilmesi düşünülebilir.